

```
-- ===== -- PROJECT NAME : BANKING
ANALYTICS SYSTEM -- TECHNOLOGY : MYSQL -- CONCEPTS : TABLES, JOINS, SUBQUERIES, VIEWS, -- STORED
PROCEDURES, TRIGGERS, -- CTEs, WINDOW FUNCTIONS, INDEXING --
=====
```

```
CREATE DATABASE BANKING_ANALYTICS;
```

```
USE BANKING_ANALYTICS;
```

```
-- ===== -- TABLE CREATION --
=====
```

```
CREATE TABLE CUSTOMERS ( CUSTOMER_ID VARCHAR(20) PRIMARY KEY, FIRST_NAME VARCHAR(50),
LAST_NAME VARCHAR(50), EMAIL VARCHAR(100), CITY VARCHAR(50), CREDIT_SCORE INT, CREATED_AT
DATETIME );
```

```
CREATE TABLE ACCOUNTS ( ACCOUNT_ID VARCHAR(20) PRIMARY KEY, CUSTOMER_ID VARCHAR(20),
ACCOUNT_TYPE VARCHAR(20), BALANCE_USD DECIMAL(15,2), OPEN_DATE DATE,
```

```
FOREIGN KEY (CUSTOMER_ID)
REFERENCES CUSTOMERS(CUSTOMER_ID)
```

```
);
```

```
CREATE TABLE LOANS ( LOAN_ID VARCHAR(20) PRIMARY KEY, CUSTOMER_ID VARCHAR(20), LOAN_AMOUNT
DECIMAL(15,2), INTEREST_RATE DECIMAL(5,2), START_DATE DATE,
```

```
FOREIGN KEY (CUSTOMER_ID)
REFERENCES CUSTOMERS(CUSTOMER_ID)
```

```
);
```

```
CREATE TABLE CARDS ( CARD_ID VARCHAR(20) PRIMARY KEY, ACCOUNT_ID VARCHAR(20), CARD_TYPE
VARCHAR(20), EXPIRATION_DATE DATE,
```

```
FOREIGN KEY (ACCOUNT_ID)
REFERENCES ACCOUNTS(ACCOUNT_ID)
```

```
);
```

```
CREATE TABLE BRANCHES ( BRANCH_ID VARCHAR(20) PRIMARY KEY, BRANCH_NAME VARCHAR(100),
MANAGER_NAME VARCHAR(100) );
```

```
CREATE TABLE MERCHANTS ( MERCHANT_ID VARCHAR(20) PRIMARY KEY, MERCHANT_NAME VARCHAR(100),  
CITY VARCHAR(50) );
```

```
-- ===== -- INDEXING --  
=====
```

```
CREATE INDEX IDX_CUSTOMER_ID ON ACCOUNTS(CUSTOMER_ID);
```

```
CREATE INDEX IDX_LOAN_CUSTOMER ON LOANS(CUSTOMER_ID);
```

```
-- ===== -- VIEW --  
=====
```

```
CREATE VIEW CUSTOMER_ACCOUNT_SUMMARY AS
```

```
SELECT C.CUSTOMER_ID, CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME, A.ACCOUNT_ID,  
A.ACCOUNT_TYPE, A.BALANCE_USD
```

```
FROM CUSTOMERS C JOIN ACCOUNTS A ON C.CUSTOMER_ID=A.CUSTOMER_ID;
```

```
-- ===== -- STORED PROCEDURE --  
=====
```

```
DELIMITER //
```

```
CREATE PROCEDURE TOP_CUSTOMERS_BY_BALANCE()
```

```
BEGIN
```

```
SELECT C.CUSTOMER_ID, CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME, A.BALANCE_USD
```

```
FROM CUSTOMERS C JOIN ACCOUNTS A ON C.CUSTOMER_ID=A.CUSTOMER_ID
```

```
ORDER BY A.BALANCE_USD DESC LIMIT 10;
```

```
END //
```

```
DELIMITER ;
```

```
CALL TOP_CUSTOMERS_BY_BALANCE();
```

```
-- ===== -- TRIGGER --  
=====
```

```
CREATE TABLE AUDIT_LOG ( LOG_ID INT AUTO_INCREMENT PRIMARY KEY, ACCOUNT_ID VARCHAR(20),  
ACTION_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

```
DELIMITER //
```

```
CREATE TRIGGER ACCOUNT_INSERT_TRIGGER
```

```
AFTER INSERT ON ACCOUNTS
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO AUDIT_LOG ( ACCOUNT_ID )
```

```
VALUES ( NEW.ACCOUNT_ID );
```

```
END //
```

```
DELIMITER ;
```

```
-- ===== -- BUSINESS QUERIES --  
=====
```

```
-- 1 TOTAL CUSTOMERS
```

```
SELECT COUNT(*) AS TOTAL_CUSTOMERS FROM CUSTOMERS;
```

```
-- 2 TOTAL ACCOUNTS
```

```
SELECT COUNT(*) AS TOTAL_ACCOUNTS FROM ACCOUNTS;
```

```
-- 3 TOTAL LOANS
```

```
SELECT COUNT(*) AS TOTAL_LOANS FROM LOANS;
```

```
-- 4 TOTAL CARDS
```

```
SELECT COUNT(*) AS TOTAL_CARDS FROM CARDS;
```

```
-- 5 TOTAL MERCHANTS
```

```
SELECT COUNT(*) AS TOTAL_MERCHANTS FROM MERCHANTS;
```

```
-- 6 CUSTOMER ACCOUNT DETAILS
```

```
SELECT C.CUSTOMER_ID, C.FIRST_NAME, A.ACCOUNT_TYPE, A.BALANCE_USD FROM CUSTOMERS C INNER  
JOIN ACCOUNTS A ON C.CUSTOMER_ID=A.CUSTOMER_ID;
```

-- 7 CUSTOMER LOAN DETAILS

```
SELECT C.CUSTOMER_ID, C.FIRST_NAME, L.LOAN_AMOUNT FROM CUSTOMERS C JOIN LOANS L ON  
C.CUSTOMER_ID=L.CUSTOMER_ID;
```

-- 8 CUSTOMER CARD DETAILS

```
SELECT C.CUSTOMER_ID, C.FIRST_NAME, CR.CARD_TYPE FROM CUSTOMERS C JOIN ACCOUNTS A ON  
C.CUSTOMER_ID=A.CUSTOMER_ID JOIN CARDS CR ON A.ACCOUNT_ID=CR.ACCOUNT_ID;
```

-- 9 ACCOUNT CARD MAPPING

```
SELECT ACCOUNT_ID, CARD_TYPE FROM CARDS;
```

-- 10 CUSTOMER LOAN SUMMARY

```
SELECT CUSTOMER_ID, SUM(LOAN_AMOUNT) TOTAL_LOAN FROM LOANS GROUP BY CUSTOMER_ID;
```

-- 11 AVERAGE ACCOUNT BALANCE

```
SELECT AVG(BALANCE_USD) FROM ACCOUNTS;
```

-- 12 MAXIMUM BALANCE

```
SELECT MAX(BALANCE_USD) FROM ACCOUNTS;
```

-- 13 MINIMUM BALANCE

```
SELECT MIN(BALANCE_USD) FROM ACCOUNTS;
```

-- 14 AVERAGE LOAN AMOUNT

```
SELECT AVG(LOAN_AMOUNT) FROM LOANS;
```

-- 15 CUSTOMER COUNT BY CITY

```
SELECT CITY, COUNT(*) TOTAL_CUSTOMERS FROM CUSTOMERS GROUP BY CITY;
```

-- 16 ACCOUNT TYPE ANALYSIS

```
SELECT ACCOUNT_TYPE, COUNT(*) TOTAL_ACCOUNTS FROM ACCOUNTS GROUP BY ACCOUNT_TYPE;
```

-- 17 AVERAGE BALANCE BY ACCOUNT TYPE

```
SELECT ACCOUNT_TYPE, AVG(BALANCE_USD) FROM ACCOUNTS GROUP BY ACCOUNT_TYPE;
```

-- 18 CITIES HAVING MORE THAN 100 CUSTOMERS

```
SELECT CITY, COUNT() TOTAL_CUSTOMERS FROM CUSTOMERS GROUP BY CITY HAVING COUNT() > 100;
```

-- 19 TOTAL LOAN BY CUSTOMER

```
SELECT CUSTOMER_ID, SUM(LOAN_AMOUNT) FROM LOANS GROUP BY CUSTOMER_ID;
```

-- 20 TOP 10 CUSTOMERS

```
SELECT C.CUSTOMER_ID, C.FIRST_NAME, A.BALANCE_USD FROM CUSTOMERS C JOIN ACCOUNTS A ON  
C.CUSTOMER_ID=A.CUSTOMER_ID ORDER BY A.BALANCE_USD DESC LIMIT 10;
```

-- 21 SUBQUERY

```
SELECT * FROM ACCOUNTS WHERE BALANCE_USD > ( SELECT AVG(BALANCE_USD) FROM ACCOUNTS );
```

-- 22 ACCOUNT WITH MAXIMUM BALANCE

```
SELECT * FROM ACCOUNTS WHERE BALANCE_USD = ( SELECT MAX(BALANCE_USD) FROM ACCOUNTS );
```

-- 23 CUSTOMERS WITH MULTIPLE ACCOUNTS

```
SELECT CUSTOMER_ID, COUNT() FROM ACCOUNTS GROUP BY CUSTOMER_ID HAVING COUNT() > 1;
```

-- 24 CTE

```
WITH HIGH_BALANCE_CUSTOMERS AS ( SELECT * FROM ACCOUNTS WHERE BALANCE_USD > 50000 )
```

```
SELECT * FROM HIGH_BALANCE_CUSTOMERS;
```

-- 25 ROW_NUMBER()

```
SELECT CUSTOMER_ID, BALANCE_USD,
```

```
ROW_NUMBER()  
OVER(ORDER BY BALANCE_USD DESC) AS ROW_NUM
```

```
FROM ACCOUNTS;
```

-- 26 RANK()

SELECT CUSTOMER_ID, BALANCE_USD,

```
RANK()  
OVER(ORDER BY BALANCE_USD DESC) AS RANK_NO
```

FROM ACCOUNTS;

-- 27 DENSE_RANK()

SELECT CUSTOMER_ID, BALANCE_USD,

```
DENSE_RANK()  
OVER(ORDER BY BALANCE_USD DESC) AS DENSE_RANK_NO
```

FROM ACCOUNTS;

-- 28 CUSTOMER SEGMENTATION

SELECT CUSTOMER_ID,

```
CASE  
  WHEN BALANCE_USD >= 100000 THEN 'PREMIUM'  
  WHEN BALANCE_USD >= 50000 THEN 'GOLD'  
  ELSE 'STANDARD'  
END AS CUSTOMER_SEGMENT
```

FROM ACCOUNTS;

-- 29 LOAN PORTFOLIO ANALYSIS

SELECT AVG(LOAN_AMOUNT) AVG_LOAN, MAX(LOAN_AMOUNT) MAX_LOAN, MIN(LOAN_AMOUNT)
MIN_LOAN FROM LOANS;

-- 30 VIEW EXECUTION

SELECT * FROM CUSTOMER_ACCOUNT_SUMMARY;

-- ===== -- END OF PROJECT --

===== -- FEATURES IMPLEMENTED: -- ✓

Database Design -- ✓ PK / FK Relationships -- ✓ Joins -- ✓ Aggregate Functions -- ✓ GROUP BY & HAVING --

✓ Subqueries -- ✓ CTEs -- ✓ Window Functions -- ✓ Views -- ✓ Stored Procedures -- ✓ Triggers -- ✓ Indexing

-- ✓ 25+ SQL Queries -- ✓ Banking Analytics Reports -- -- PROJECT SUITABLE FOR: -- SQL Developer -- SQL
Programmer -- Database Developer -- Data Analyst --
=====